ETHNUS™

Explore | Expand | Enrich

<Codemithra />™

K-ARRAY HEAP

K-ARRAY HEAP

EXPLANATION

The d-ary heap or d-heap or k-ary heap is a priority queue data structure, a generalization of the binary heap in which the nodes have d children instead of 2.

K-ARRAY HEAP

EXPLANATION

Properties

Nearly complete binary tree, with all levels having maximum number of nodes except the last, which is filled in left to right manner.

It can be divided into two categories:

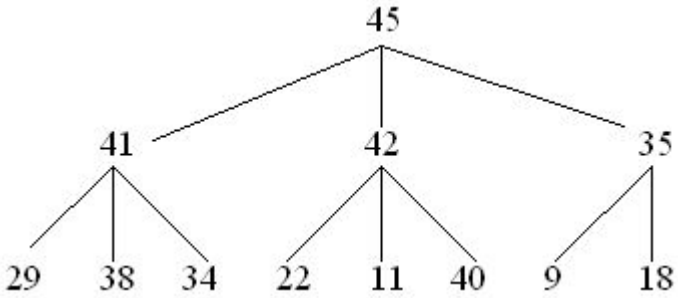
(a) Max k-ary heap: Key at root is greater than all descendants and same is recursively true for all nodes

(b) Min k-ary heap: Key at root is lesser than all descendants and same is recursively true for all nodes

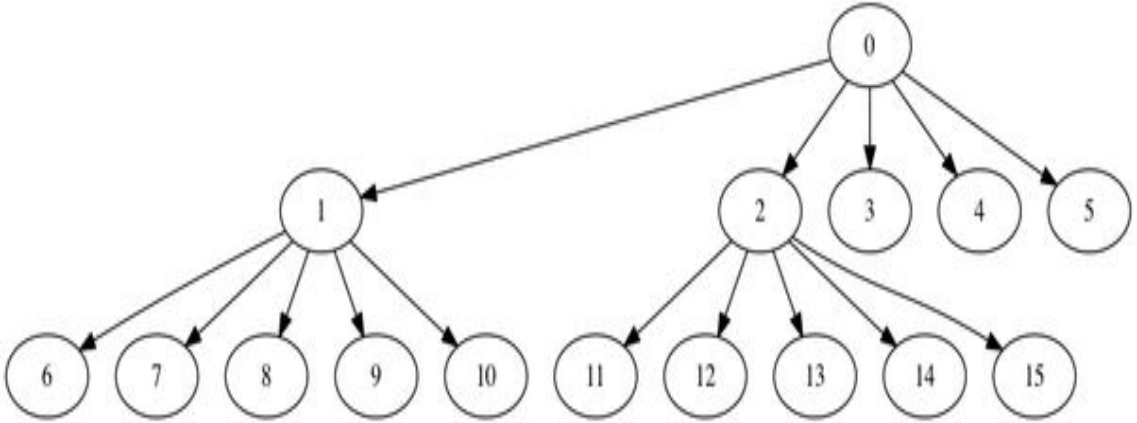
K-ARRAY HEAP

EXAMPLE

3-ary max heap



5-ary min heap



K-ARRAY HEAP

IMPLEMENTATION

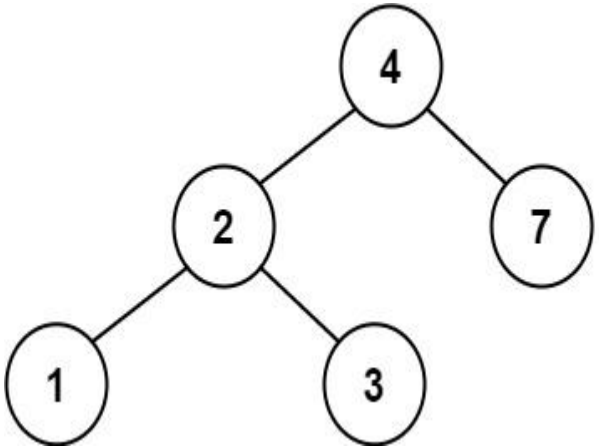
Assuming 0 based indexing of array, an array represents a K-ary heap such that for any node we consider:

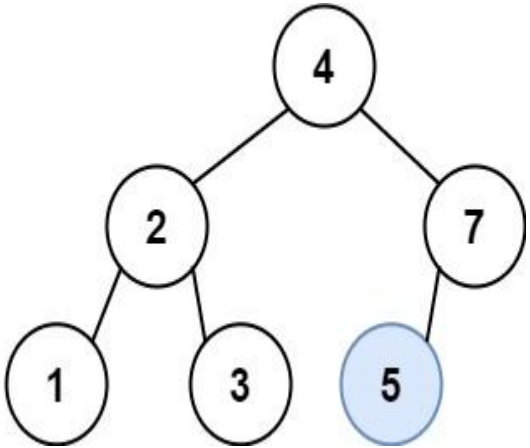
- Parent of the node at index i (except root node) is located at index (i-1)/k
- Children of the node at index i are at indices (k*i)+1 , (k*i)+2 …. (k*i)+k
- The last non-leaf node of a heap of size n is located at index (n-2)/k

K-ARRAY HEAP

IMPLEMENTATION

1. insert() : Inserting a node into the heap

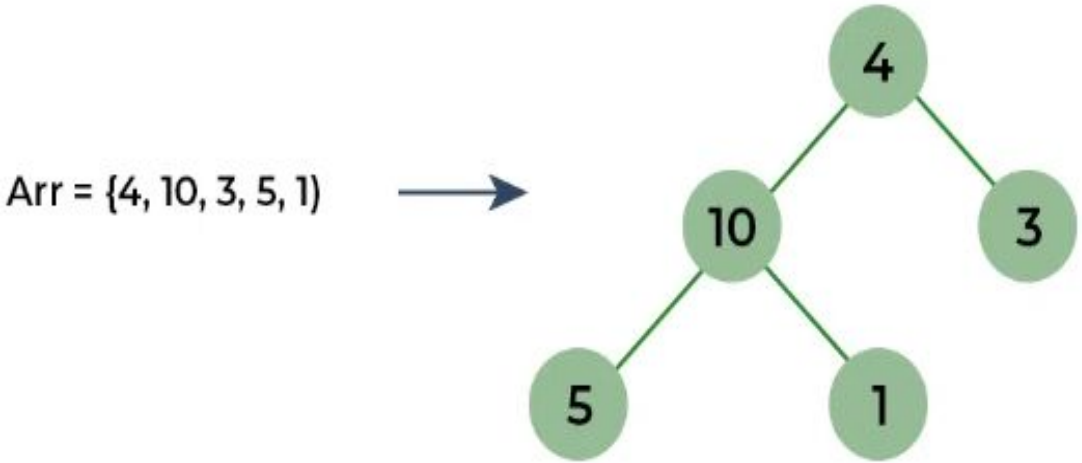




K-ARRAY HEAP

IMPLEMENTATION

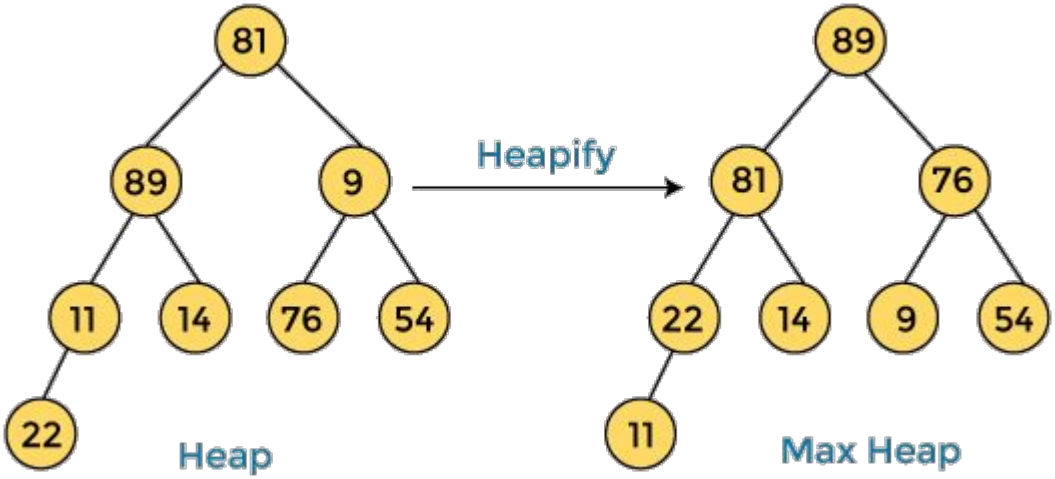
buildHeap() : Builds a heap from an input array.



K-ARRAY HEAP

IMPLEMENTATION

restoreDown() /maxHeapify() : Used to maintain heap property



K-ARRAY HEAP

IMPLEMENTATION

extractMax() : Extacting the root node.

K-ARRAY HEAP

```
import java.util.Scanner;
import java.util.Arrays;
import java.util.NoSuchElementException;
class DaryHeap
{
    private int d;
    private int heapSize;
    private int[] heap;
    public DaryHeap(int capacity, int numChild)
    {
        heapSize = 0;
        d = numChild;
        heap = new int[capacity + 1];
        Arrays.fill(heap, -1);
    }
    public boolean isEmpty( )
    {
        return heapSize == 0;
    }
    public boolean isFull( )
    {
        return heapSize == heap.length;
    }
public void clear( )
    {
        heapSize = 0; }
```

```
private int parent(int i)
    {
        return (i - 1)/d;
    }
    private int kthChild(int i, int k)
    {
        return d * i + k;
    }
public void insert(int x)
    {
        if (isFull( ) )
            throw new
NoSuchElementException("Overflow Exception");
        heap[heapSize++] = x;
        heapifyUp(heapSize - 1);
    }
    public int findMin( )
    {
        if (isEmpty() )
            throw new
NoSuchElementException("Underflow Exception");
        return heap[0];
    }
 public int delete(int ind)
    {
```

K-ARRAY HEAP

```
if (isEmpty() )
            throw new NoSuchElementException("Underflow
Exception");
        int keyItem = heap[ind];
        heap[ind] = heap[heapSize - 1];
        heapSize--;
        heapifyDown(ind);
        return keyItem;
    }
    private void heapifyUp(int childInd)
    {
        int tmp = heap[childInd];
        while (childInd > 0 && tmp <
heap[parent(childInd)])
        {
            heap[childInd] = heap[ parent(childInd) ];
            childInd = parent(childInd);
        }
        heap[childInd] = tmp;
    }
    private void heapifyDown(int ind)
    {
        int child;
        int tmp = heap[ ind ];
```

```
while (kthChild(ind, 1) < heapSize)
        {
            child = minChild(ind);
            if (heap[child] < tmp)
                heap[ind] = heap[child];
            else
                break;
            ind = child;
        }
        heap[ind] = tmp;
    }
    private int minChild(int ind)
    {
        int bestChild = kthChild(ind, 1);
        int k = 2;
        int pos = kthChild(ind, k);
        while ((k <= d) && (pos < heapSize))
        {
            if (heap[pos] < heap[bestChild])
                bestChild = pos;
            pos = kthChild(ind, k++);
        }
        return bestChild;
    }
```

K-ARRAY HEAP

```
public void printHeap()
    {
        System.out.print("\nHeap = ");
        for (int i = 0; i < heapSize; i++)
            System.out.print(heap[i] +" ");
        System.out.println();
    }
}
public class DaryHeapTest
{
    public static void main(String[] args)
    {
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter size and D of D-ary
Heap");
        DaryHeap dh = new DaryHeap(scan.nextInt(),
scan.nextInt() );
        char ch;
        do
        {
            System.out.println("\nD-ary Heap
Operations\n");
            System.out.println("1. insert ");
            System.out.println("2. delete");
            System.out.println("3. check full");
```

```
System.out.println("4. check empty");
            System.out.println("5. clear");
            boolean chk;
            int choice = scan.nextInt();
            switch (choice)
            {
            case 1 :
                try
                {
                    System.out.println("Enter
integer element to insert");
                    dh.insert( scan.nextInt() );
                }
                catch (Exception e)
                {
               System.out.println(e.getMessage() );
                }
                break;
            case 2 :
                try
                {
              System.out.println("Enter delete
position");
                    dh.delete(scan.nextInt() - 1);
                }
```

K-ARRAY HEAP

```
 catch (Exception e)
                {
             System.out.println(e.getMessage()
);
                }
                break;
            case 3 :
                System.out.println("Full status
= "+ dh.isFull());
                break;
            case 4 :
                System.out.println("Empty status
= "+ dh.isEmpty());
                break;
            case 5 :
                dh.clear();
                System.out.println("Heap
Cleared\n");
                break;
```

```
 default :
                System.out.println("Wrong Entry
\n ");
                break;
            }
            /** Display heap **/
            dh.printHeap();

            System.out.println("\nDo you want to
continue (Type y or n) \n");
            ch = scan.next().charAt(0);
        } while (ch == 'Y'|| ch == 'y');
    }
}
```

INTERVIEW QUESTIONS

1.What is a K-array Heap?

Answer: A K-array is an array data structure where each element can have up to K distinct values.

INTERVIEW QUESTIONS

2. How is a K-ary Heap different from a Binary Heap?

Answer: In a Binary Heap, each node has at most two children (left and right), while in a K-ary Heap, each node can have up to K children.

INTERVIEW QUESTIONS

3. What is the advantage of using a K-ary Heap over a Binary Heap?

Answer: K-ary Heaps can be more space-efficient than Binary Heaps in certain situations since each node has more children, reducing the overall height of the tree.

INTERVIEW QUESTIONS

4. Explain the operations of insertion and deletion in a K-ary Heap.

Answer: Insertion involves adding a new element to the heap and then restoring the heap property by performing the "heapify-up" operation. Deletion involves removing the root element, replacing it with the last element, and then restoring the heap property by performing the "heapify-down" operation.

INTERVIEW QUESTIONS

5. How do you represent a K-ary Heap in an array?

Answer: In a K-ary Heap, if a node is at index i, its children are at indices Ki + 1, Ki + 2, ..., K*i + K.

https://learn.codemithra.com

ETHNUS™
Explore | Expand | Enrich

THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com